OBJECT-ORIENTED PROGRAMMING · PROJECT REPORT

Bookstore Inventory Manager

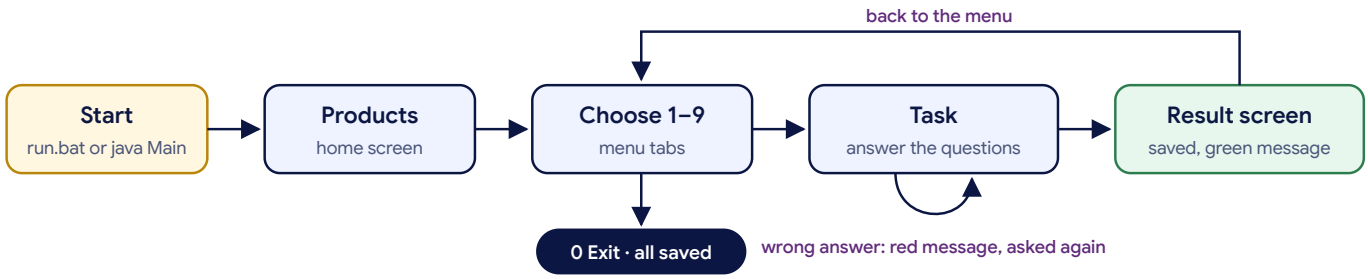
Java 17 console application · 4 files, 617 lines · github.com/AtlasRoX/Bookstore-JAVA

A program for a small bookstore that sells books and stationery. Staff can add, search and restock products, change prices, sell with an optional 5% member discount, refund sales, and see sales history and a report. Every change is saved automatically to `bookstore.dat`. A wrong answer is explained in red and asked again; typing `cancel` stops a task.

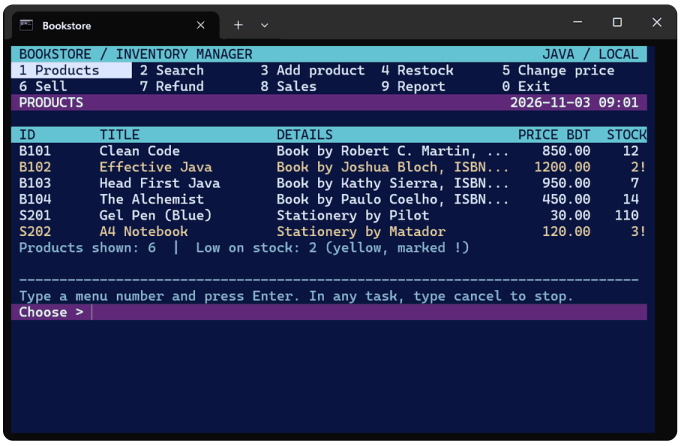
File	Job
`Product.java`	Interfaces Item, Trackable, Promotable; abstract class Product; subclasses Book and Stationery
`Bookstore.java`	Shop rules (add, restock, sell, refund, search), saving and loading; class Sale (one receipt)
`BookstoreUI.java`	Everything on screen: colors, menu tabs, tables, receipt, questions
`Main.java`	Starts the program, runs the menu and one method per task

Run: double-click `run.bat` (opens an 80 × 24 window), or `javac *.java` then `java Main`. **Tested:** 30/30 automated test cases passed; 50 random-input sessions, 0 crashes.

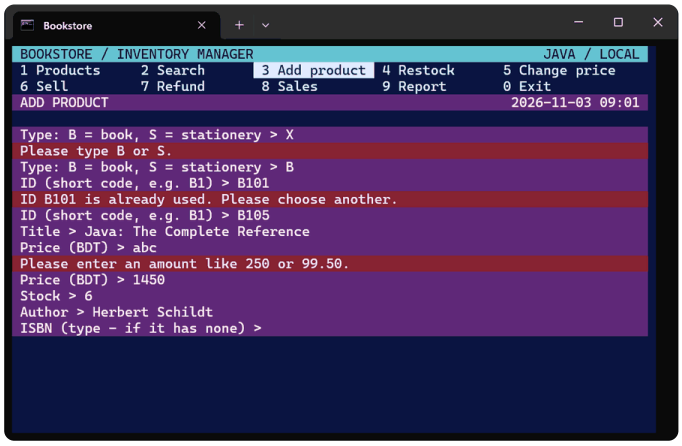
User Flow



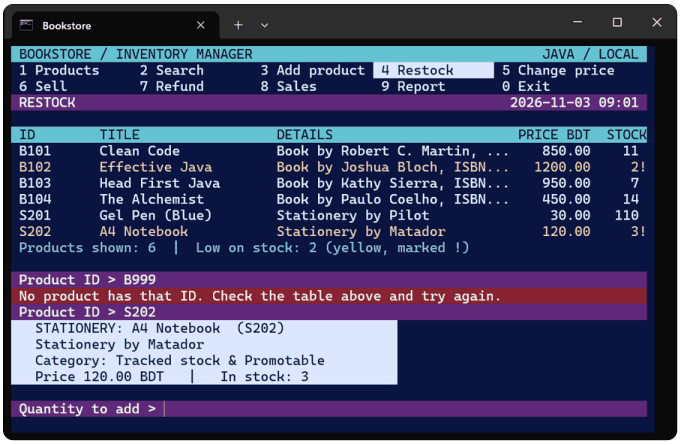
Screens



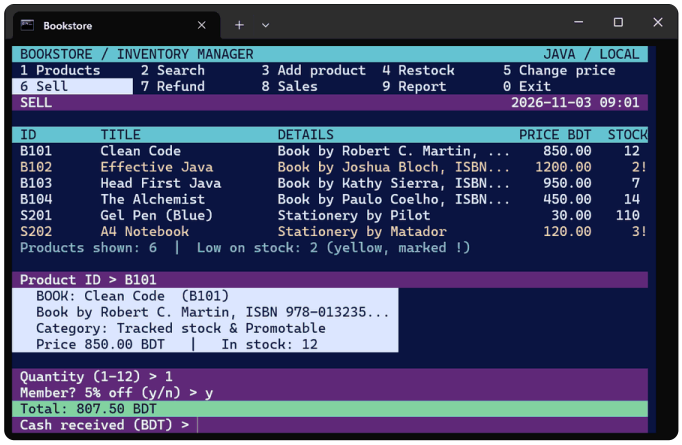
1 Products — all products; low stock (≤ 3) is yellow with "!".



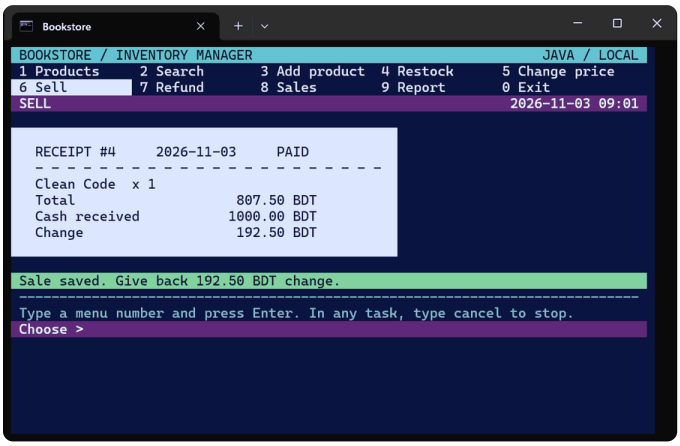
3 Add product — each wrong answer is explained and asked again.



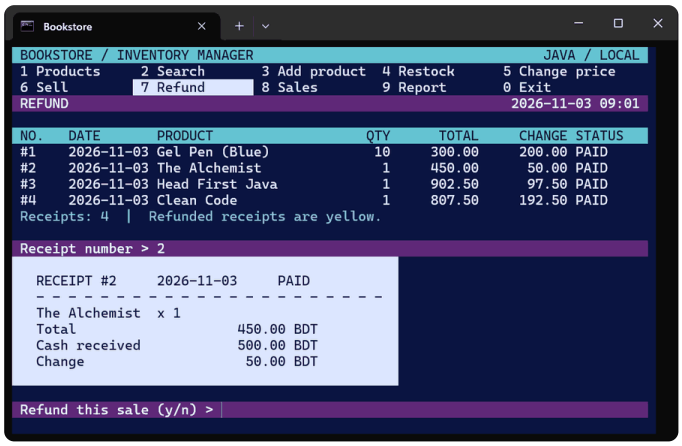
4 Restock — unknown ID refused; the card shows type, category and stock.



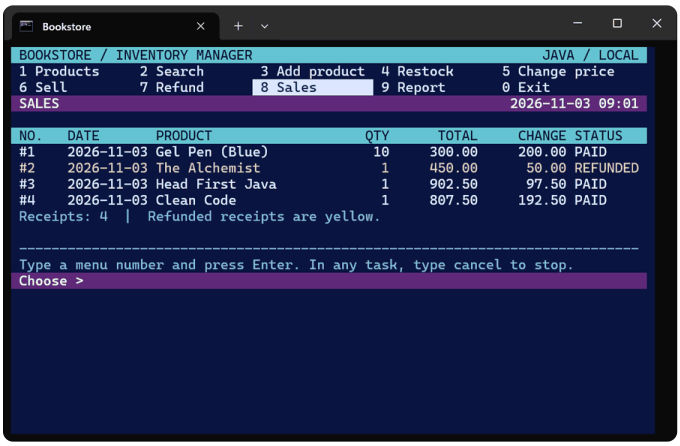
6 Sell — quantity range, 5% member discount, total.



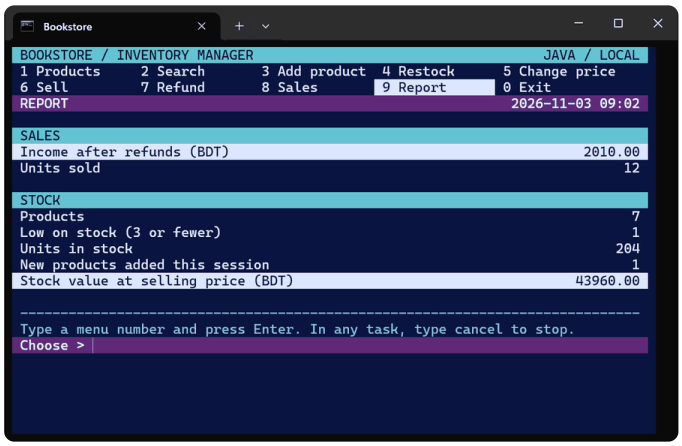
6 Sell — receipt with the change to give back.



7 Refund — choose a receipt and confirm; stock goes back.



8 Sales — all receipts; refunded ones are yellow.



9 Report — income after refunds, stock, products added this session.

OOP Concepts Used

1 Abstraction

`Product` is abstract, so only `Book` and `Stationery` objects are created. Every product must describe itself through the abstract methods.

Product.java lines 27–83

```
27  public abstract class Product implements Trackable, Promotable, Serializable {
…
82  public abstract String getType();
83  public abstract String getDetails();
```

2 Encapsulation

Fields are `private`; other classes use getters and setters, and the setters refuse negative values.

Product.java lines 40–64

```
40  private String id, title;
41  private double price;
42  private int stock;
…
62  public void setPrice(double price) {
63      if (price >= 0) this.price = price;
64  }
```

3 Inheritance

`Book` and `Stationery` (subclasses) inherit id, title, price, stock and all their methods from `Product` (superclass) and add only their own fields.

Product.java lines 87–116

```
87  class Book extends Product {
…
89  private String author, isbn;
…
114 class Stationery extends Product {
…
116 private String brand;
```

4 Polymorphism

The product table calls `p.getDetails()` on a `Product` variable; at run time Java runs Book's or Stationery's version (runtime polymorphism). Overloading (6) is compile-time polymorphism.

BookstoreUI.java lines 135–144

```
135 for (Product p : list) {
…
144 fit(p.getId(), 9), fit(p.getTitle(), 21), fit(p.getDetails(), 29), p.getPrice(), stock));
```

5 Static vs non-static

`CURRENCY`, `createdCount` and `getCreatedCount()` belong to the class and are shared; the static block runs once when the class loads. The Report shows `Product.getCreatedCount()`. Fields like `price` are non-static: each object has its own.

Product.java lines 31–53

```
31  public static final String CURRENCY;
32  private static int createdCount = 0; // products created while the program runs (shown in the Report)
…
35  static {
36      CURRENCY = "BDT";
37  }
…
49  createdCount++;
…
53  public static int getCreatedCount() { return createdCount; }
```

6 Method overloading

Same name, different parameters: a `Book` is created with or without an ISBN (the user types "-" for none), and `restock` takes an ID or a product.

Product.java lines 92–99

```
92  public Book(String id, String title, double price, int stock, String author, String isbn) {
…
99  public Book(String id, String title, double price, int stock, String author) {
```

Bookstore.java lines 46–50

```
46  public void restock(String id, int quantity) {
…
50  public void restock(Product p, int quantity) {
```

7 Method overriding

Each subclass gives its own `getType()` and `getDetails()`, marked `@Override`.

Product.java lines 106–129

```
106 @Override
107 public String getType() { return "BOOK"; }
109 @Override
110 public String getDetails() { return "Book by " + author + ", ISBN " + isbn; }
    …
125 @Override
126 public String getType() { return "STATIONERY"; }
128 @Override
129 public String getDetails() { return "Stationery by " + brand; }
```

8 this keyword

`this.id` is the object's field, `id` the parameter. `this(...)` calls the other constructor and fills the missing ISBN.

Product.java lines 45–100

```
 45 this.id = id; // THIS: this.id is the object's field, id is the parameter
    …
100 this(id, title, price, stock, author, "N/A");
```

9 super keyword

Subclass constructors first call the `Product` constructor with `super(...)`.

Product.java lines 118–119

```
118 public Stationery(String id, String title, double price, int stock, String brand) {
119 super(id, title, price, stock); // SUPER: runs the Product constructor first
```

10 Subclass vs superclass reference

While a book is created, `Book book` can call `getAuthor()`. Then `product = book` stores it as a `Product` (upcasting), like every item in the inventory.

Main.java lines 73–88

```
 73 Product product; // SUPERCLASS reference: can hold a Book or a Stationery
    …
 78 Book book; // SUBCLASS reference
    …
 81 extra = " by " + book.getAuthor(); // getAuthor() works only through the Book reference
 82 product = book;                    // UPCASTING: the Book is stored as a Product
    …
 88 store.add(product);
```

11 Diamond problem resolution

`Trackable` and `Promotable` both extend `Item` with their own default `getCategory()`. `Product` implements both, so Java forces it to choose; it calls both with `Interface.super`. The result appears on the product card.

Product.java lines 11–23

```
11 interface Item {
12     default String getCategory() { return "Item"; }
13 }
15 interface Trackable extends Item {
16     @Override
17     default String getCategory() { return "Tracked stock"; }
18 }
20 interface Promotable extends Item {
21     @Override
22     default String getCategory() { return "Promotable"; }
23 }
```

Product.java lines 77–79

```
77 public String getCategory() {
78     return Trackable.super.getCategory() + " & " + Promotable.super.getCategory();
79 }
```